Entra Verified ID — Verifiable Credentials

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Entra Verified ID — Verifiable Credentials 5
What you get . . . 5
What this delivers . . . 5
Scope boundary . . . 6
The standard deployment model . . . 6
Outcome . . . 6
1. Solution Overview — Entra Verified ID — Verifiable Credentials 6
What Entra Verified ID is . . . 6
How it works . . . 6
How this fits the rest of the identity catalog (gap-only) . . . 7
What you can control . . . 7
Benefits . . . 8
Limitations (be honest with buyers) . . . 8
Authentication and connections used . . . 8
The standard deployment model . . . 9
2. Requirements Guide — Entra Verified ID — Verifiable Credentials 9
2.1 Verified ID availability and licensing — verify . . . 9
2.2 Administrative roles for the operator . . . 9
2.3 Azure Key Vault for signing/encryption keys . . . 10
2.4 A verified, controllable domain . . . 10
2.5 App registration for the Request Service API . . . 10
2.6 Microsoft Authenticator wallet for the pilot holder . . . 10
2.7 A pilot credential design and a relying-party endpoint . . . 10
2.8 Microsoft Graph PowerShell SDK . . . 11
Checklist . . . 11
Global Requirements — Entra ID Solutions (category-level) 11
G1. Microsoft Entra ID tenant . . . 11
G2. Entra ID licensing (P1, P2 where noted) . . . 12
G3. Administrative roles . . . 12
G4. Break-glass (emergency access) accounts . . . 12
G5. Healthy on-premises Active Directory . . . 12
G6. An Entra Connect host server . . . 12
G7. UPN / domain hygiene . . . 12
G8. Network access to Microsoft cloud endpoints . . . 12
3. Implementation Guide — Entra Verified ID — Verifiable Credentials 13

Step 1 — Assess prerequisites . . . 13
Step 2 — Create/prepare the Azure Key Vault . . . 13
Step 3 — Onboard the Verified ID service and set up the trust system . . . 13
Step 4 — Verify your domain . . . 14
Step 5 — Design the credential type (rules + display) . . . 14
Step 6 — Register the Request Service API app . . . 14
Step 7 — Issue a pilot credential (issuance flow) . . . 14
Step 8 — Verify the pilot credential (verification flow) . . . 15
Step 9 — Revoke and confirm governance . . . 15
Step 10 — Validate on the pilot scope . . . 15
Step 11 — Expand to production . . . 15
5. Testing Guide — Entra Verified ID — Verifiable Credentials 15
Test 0 — Service and prerequisites confirmed . . . 16
Test 1 — Domain is verified (well-known resolves) . . . 16
Test 2 — Credential type is published . . . 16
Test 3 — Issuance request builds safely (no submit) . . . 16
Test 4 — Credential issues to the wallet . . . 16
Test 5 — Verification returns verified claims . . . 16
Test 6 — Revocation is honored . . . 16
Test 7 — Issued/revoked report . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Entra Verified ID — Verifiable Credentials 17
A. Onboarding / service setup problems . . . 17
B. Domain verification problems . . . 18
C. Request Service app / API problems . . . 18
D. Issuance problems . . . 18
E. Verification problems . . . 19
F. Governance / lifecycle . . . 19
G. Diagnostics quick reference . . . 19
H. When to escalate . . . 19
7. Best Practices — Entra Verified ID — Verifiable Credentials 20
Where to go next (adjacent catalog areas) . . . 21
8. Deliverables Summary — Entra Verified ID — Verifiable Credentials 21
Documentation . . . 21
PowerShell scripts (Microsoft Graph + Verified ID Request Service API) . . . 22
What the buyer receives . . . 22

Outcome 22
Appendix A — PowerShell Scripts 24

Executive Summary — Entra Verified ID — Verifiable Credentials

White-label · Entra ID Solutions

Manual identity and attribute verification — proving who a new hire is, that someone is still employed, or that a person is entitled to a sensitive resource — is slow, inconsistent, and phishable. It leaves paper trails and one-off email confirmations instead of proof. Microsoft Entra Verified ID replaces that with **verifiable credentials**: your organization issues tamper-evident digital credentials to a person's wallet, and any relying party can cryptographically verify them — without calling you, and without the person over-sharing. The credential is issued once, held by the person, verified on demand, and revocable when it should no longer be trusted.

This solution is a full **implementation** product, not a talk-through. It is written for an administrator with no prior verifiable-credentials experience — exact, ordered, click-by-click steps in the Microsoft Entra admin center and the Verified ID service — and it stands up the whole capability end to end: the Verified ID service and its trust system, a credential type with its rules and display, a working **issuance flow** (issue an employee/verification credential to the Microsoft Authenticator wallet), a working **verification flow** (a relying party checks the credential), and the governance around it (revocation, lifecycle, and privacy).

What you get

• A plain-language Solution Overview of Verified ID: the trust system, the decentralized identifier (DID) and domain verification, credential types, rules and display definitions, the issuance and verification flows, and the Request Service API — with the feature/preview and licensing items flagged *verify*.

• A Requirements Guide with Verify / Implement-if-missing for every solution-specific prerequisite (the category-wide ones live in the separate global-requirements file), including the Azure Key Vault, domain, and app-registration prerequisites Verified ID needs.

• An Implementation Guide of precise, ordered steps: onboard the Verified ID service, verify your domain, design a credential type (rules + display), register an app for the Request Service API, issue a pilot credential to a wallet, verify it from a relying party, and revoke it — piloted before rollout.

• Five PowerShell scripts (Microsoft Graph + Verified ID Request Service API) for service/authority discovery, trust-and-domain validation, an issued/revoked credential report, a safe issuance-request builder (builds the request and only submits when told), and a safe presentation (verification) request builder.

• Testing and Troubleshooting guides, plus Best Practices and a Deliverables Summary, for a confident rollout.

What this delivers

A working Verified ID capability: an onboarded issuer with a verified domain and a published credential type; the ability to issue a credential (for example a Verified Employee credential) to a person's wallet through a QR-code/deep-link flow; the ability for a relying party to request and cryptographically verify that credential; and the governance to revoke a credential and manage its lifecycle — all proven on a pilot before you scale. Privacy is by design: the holder consents to each presentation and shares only the claims requested.

Scope boundary

This solution stands up Verified ID and one issuance and one verification flow end to end. It does **not** re-implement Conditional Access, entitlement management, or hybrid identity — those are separate catalog areas it can plug into (for example, using a Verified ID presentation as a signal — *verify current availability*). Decisions about which credentials are authoritative, what claims are lawful to collect, and acceptable use belong to your compliance, legal, and security functions; this guide provides the technical controls, not the decisions.

The standard deployment model

Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Issue and verify tamper-evident digital credentials for onboarding, proof of employment, and access — privacy-respecting by design, revocable, and standards-based — configured correctly and proven on a pilot before you scale.

1. Solution Overview — Entra Verified ID — Verifiable Credentials

What Entra Verified ID is

Microsoft Entra Verified ID is Microsoft's implementation of **verifiable credentials** — an open standard (W3C Verifiable Credentials and Decentralized Identifiers) for issuing and verifying tamper-evident digital claims about a person or organization. Instead of verifying identity or attributes by phone, email, or a portal look-up, an **issuer** cryptographically signs a credential and gives it to the subject; the subject stores it in a **wallet** (Microsoft Authenticator); and any **verifier** (relying party) can request that credential and confirm, mathematically, that it was issued by a trusted issuer, has not been tampered with, and has not been revoked — without contacting the issuer.

Three roles define the model:

- **Issuer** — your organization signs and issues credentials (for example a *Verified Employee*

credential asserting name, job title, and that the person is employed).

- **Holder** — the person who receives the credential into their wallet and consents to present it.
- **Verifier / relying party** — an app or service that requests a credential and verifies it to make

a decision (grant access, complete onboarding, prove eligibility).

How it works

1. You **onboard the Verified ID service** in your Entra tenant. Onboarding provisions a **trust

system **and a** decentralized identifier (DID) **for your organization — the cryptographic identity that signs your credentials — with signing keys held in** Azure Key Vault**.

1. You **verify your domain** (for example `contoso.com`). Verified ID publishes a

`did-configuration.json` at your domain's `/.well-known/` path linking your DID to your domain, so verifiers see a human-readable, domain-anchored issuer instead of an opaque identifier.

1. You **design a credential type**: a **rules definition** (what claims the credential carries and how

the subject is attested — for example by signing in with their Entra account, so claims come from the directory) and a **display definition** (how the credential looks in the wallet — name, logo, colors, claim labels).

1. **Issuance flow**: your issuance app calls the **Request Service API** to create an issuance

request; the person scans a QR code or follows a deep link, signs in / attests as the rules require, consents, and the credential lands in their Authenticator wallet.

1. **Verification flow**: a relying party calls the Request Service API to create a **presentation

request** for a credential type; the person is shown a QR code / deep link, consents in the wallet, and presents the credential; the relying party's callback receives the verified claims and the cryptographic proof.

1. **Governance**: you can **revoke** an issued credential (so verifiers that check status reject it),

manage credential **lifecycle** (expiry, re-issuance), and rely on the model's **privacy** properties — the holder consents to each presentation and shares only the requested claims (selective disclosure), and the issuer is not contacted during verification.

How this fits the rest of the identity catalog (gap-only)

This solution stands up **Verified ID** end to end and references adjacent capabilities rather than duplicating them:

• **Conditional Access** (`ENTZTCA`) — Verified ID can serve as a signal/authentication context in

some scenarios; this solution builds the credential capability and points to `ENTZTCA` for the CA baseline. Using a Verified ID presentation inside a Conditional Access flow is an evolving area — *verify current availability in your tenant*.

• **Entra ID Governance / entitlement management** (`ENTIGA`) — Verified ID can support onboarding and

access decisions that governance then enforces; the governance build-out lives in `ENTIGA`.

• **External Identities / B2B** (`ENTB2B`) — Verified ID is a way to onboard and prove external users;

the cross-tenant collaboration plane is `ENTB2B`.

• **Privileged Identity Management** (`ENTPIM`), **passwordless / passkey** (`ENTPASSKEY`), **hybrid

identity, and** enterprise-app governance** are separate catalog areas referenced where relevant.

What you can control

• The Verified ID service onboarding, the organization DID and its Key Vault signing keys, and domain

verification (`did-configuration.json`).

• Credential types — rules definitions (claims and attestation method: ID-token/Entra sign-in, self-attested input, or presentation of another credential) and display definitions (branding and claim labels).

• The issuance flow (which app issues, to whom, with which PIN/security options).

• The verification flow (which credential types are accepted, which claims are requested, the callback endpoint).

• Governance — credential revocation, expiry/lifecycle, and the privacy posture (claims requested, consent).

Benefits

• Tamper-evident and phishing-resistant — a credential is cryptographically signed; a verifier proves authenticity mathematically, not by trusting a screenshot or an email.

• Verify without a callback — verifiers confirm a credential without contacting the issuer, so verification is fast and works across organizational boundaries.

• Privacy by design — the holder consents to each presentation and shares only the requested claims; the issuer does not learn where the credential is used.

• Portable and reusable — one issued credential can be presented to many verifiers for onboarding, proof of employment, help-desk identity proofing, and access.

• Revocable — a credential can be revoked when it should no longer be trusted.

Limitations (be honest with buyers)

• Verified ID requires **Azure Key Vault**, a **verified domain** you control, and an **app registration** with permission to call the Request Service API; these prerequisites must be in place before issuance works (see requirements).

• Licensing and availability: Verified ID capabilities and any premium/preview features may depend on Entra ID licensing and may be in **preview** or vary by cloud — **the Verified ID feature set and preview status must be verified for your tenant** (*verify current availability in your tenant*).

• The holder needs a **wallet** (Microsoft Authenticator) and connectivity to receive and present credentials.

• Verified ID proves *what the issuer asserted*; it does not, by itself, decide whether a given credential is authoritative for a given decision — that governance decision belongs to you.

• This solution does not make regulatory, data-classification, lawful-claims, or acceptable-use decisions — those belong to your compliance, legal, and security functions.

Authentication and connections used

• Portal work is done in the Microsoft Entra admin center (`https://entra.microsoft.com`, under Verified ID) and the Azure portal (`https://portal.azure.com`) for Key Vault.

• Discovery/validation scripts use the **Microsoft Graph PowerShell SDK** (`Connect-MgGraph`) with read-only scopes.

• The issuance and verification scripts call the **Verified ID Request Service API** using an app-registration token (client credentials); those scripts **build the request and only submit when explicitly told**, so a dry run never issues or requests anything.

The standard deployment model

> *Assess prerequisites and tenant readiness -> Configure the Verified ID service, domain, credential type, and Request Service app -> Pilot: issue one credential to a wallet and verify it from a relying party -> Validate issuance, verification, and revocation -> Expand to more credential types and relying parties. pattern: Assess -> Configure -> Pilot -> Validate -> Expand.*

2. Requirements Guide — Entra Verified ID — Verifiable Credentials

Verify every requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites (tenant, base licensing, admin roles, break-glass accounts, network access) are in `global-requirements-entra-id-solutions.md` and are not repeated here.

> *The Verified ID feature set, premium options, and any preview capabilities are flagged **verify** — confirm current availability and the feature-to-license mapping in *your* tenant before you rely on them; Microsoft changes these over time.*

2.1 Verified ID availability and licensing — verify

Description: Entra Verified ID available in your tenant, with any licensing it requires for the features you use. Why: Verified ID is administered from the Entra admin center and depends on the service being available and appropriately licensed; some capabilities may be premium or preview. Verify: Entra -> `Verified ID` appears in the admin center; you can start onboarding. Run `ENTVID-Discovery.ps1`. *Verify current Verified ID availability and licensing in your tenant.* Implement: Confirm the tenant has Verified ID; assign any required Entra ID licensing (commonly Entra ID P1/P2 for the identity plane) to the administrators; check Microsoft's current Verified ID documentation for the precise licensing and any preview flags at the time you deploy.

2.2 Administrative roles for the operator

Description: A `Global Administrator` for the one-time Verified ID onboarding and consent, and an `Authentication Policy Administrator` (or equivalent) plus Key Vault permissions for ongoing configuration. Why: Onboarding the Verified ID service and granting the app permissions requires elevated rights; Key Vault key operations require Key Vault access. Verify: Entra -> `Roles and administrators`; confirm the operator can onboard Verified ID and administer the Request Service app. Run `ENTVID-Discovery.ps1`. Implement: Assign the required roles for onboarding; grant the operator

Key Vault access (see 2.3); reserve `Global Administrator` for the one-time onboarding consent.

2.3 Azure Key Vault for signing/encryption keys

Description: An Azure Key Vault in the same tenant, with access for the Verified ID service to create and use signing and encryption keys. Why: Verified ID stores the keys that sign your credentials and your DID in Key Vault; without it the service cannot be onboarded. Verify: Azure portal (`portal.azure.com`) -> the Key Vault exists; the operator (and the Verified ID service principal) have the required key permissions (get, sign, and key create as documented). Run `ENTVID-Validate.ps1`. Implement: Create a Key Vault; grant the Verified ID onboarding process and operator the documented key permissions (access policies or Azure RBAC key roles). Follow the current Verified ID onboarding guidance for the exact permission set.

2.4 A verified, controllable domain

Description: A public domain you control (for example `contoso.com`) where you can publish a file at `/.well-known/did-configuration.json`. Why: Domain verification links your DID to a human-readable domain so verifiers trust the issuer; it requires publishing a well-known configuration file at your domain. Verify: You can host content at `https://contoso.com/.well-known/did-configuration.json`; the domain is reachable over HTTPS. Run `ENTVID-Validate.ps1` after onboarding to confirm the file resolves. Implement: Choose the domain that will represent your issuer; ensure you can publish the `did-configuration.json` Verified ID generates to its `/.well-known/` path (web server or hosting you control). Avoid using a domain you cannot host files on.

2.5 App registration for the Request Service API

Description: A Microsoft Entra app registration with a client secret/certificate and permission to call the Verified ID **Request Service API**. Why: The issuance and verification flows (and the two request-builder scripts) authenticate to the Request Service API with this app's client credentials. Verify: Entra -> `App registrations`; the app exists with the Verified ID Request Service permission granted (admin-consented) and a valid secret/certificate. Run `ENTVID-Validate.ps1`. *Verify the current API permission name/ID in your tenant.* Implement: Register an app; add and admin-consent the Verified ID Request Service API permission; create a client secret or (preferred) a certificate; record the client ID, tenant ID, and the authority/issuer authority for the scripts. Store the secret securely — never in the repo or in source control.

2.6 Microsoft Authenticator wallet for the pilot holder

Description: A test user with Microsoft Authenticator installed to receive and present the pilot credential. Why: Credentials are issued to and presented from a wallet; without one you cannot complete an issuance or verification. Verify: The pilot user has Authenticator installed and can scan a QR code / open a deep link. Implement: Install Microsoft Authenticator on the pilot user's device before the pilot issuance.

2.7 A pilot credential design and a relying-party endpoint

Description: A defined pilot credential type (for example *Verified Employee* with name, job title, employment status) and a callback endpoint for the verification flow. Why: Issuance needs a credential

type with rules/display; verification needs a callback URL where the Request Service posts the verified result. Verify: You have decided the pilot credential's claims and attestation method, and you have an HTTPS callback endpoint reachable by the Request Service. Run `ENTVID-New-PresentationRequest.ps1` (build-only) to confirm the request payload assembles. Implement: Design the pilot credential (start with the built-in Verified Employee pattern where suitable) and stand up (or reuse) an HTTPS callback endpoint for the verification result; keep the pilot to one credential type and one relying party.

2.8 Microsoft Graph PowerShell SDK

Description: The Microsoft Graph PowerShell SDK on the operator workstation. Why: The discovery/validation/report scripts call Microsoft Graph and Verified ID admin/authority data. Verify: `Get-Module Microsoft.Graph.Authentication -ListAvailable` returns a version; `Connect-MgGraph` connects to `graph.microsoft.com`. Run `ENTVID-Discovery.ps1`. Implement: `Install-Module Microsoft.Graph -Scope CurrentUser`.

Checklist

- Verified ID available + licensing confirmed — `ENTVID-Discovery.ps1` · *verify*
- Operator holds onboarding/admin roles + Key Vault access
- Azure Key Vault created with the documented key permissions — `ENTVID-Validate.ps1`
- A verified, host-controllable domain for `did-configuration.json` — `ENTVID-Validate.ps1`
- App registration with Request Service API permission + secret/cert — `ENTVID-Validate.ps1` · *verify*
- Pilot holder has Microsoft Authenticator
- Pilot credential design + relying-party callback endpoint decided
- Graph PowerShell SDK present; `Connect-MgGraph` connects

Global Requirements — Entra ID Solutions (category-level)

These requirements apply to every solution in the Entra ID Solutions category. They are published once at the category level so individual listings stay small. Confirm these before deploying any identity solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *The identity plane is high risk. A mistake here breaks sign-in for everyone, so every change is staged and a break-glass account is kept ready before you touch anything.*

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID tenant. Why: It is the cloud identity backbone every other identity service builds on. Verify: Sign in to `https://entra.microsoft.com` as an administrator. Implement: Create a Microsoft 365 or Azure subscription, which provisions a tenant.

G2. Entra ID licensing (P1, P2 where noted)

Description: Microsoft Entra ID P1 (or P2 for risk-based and PIM features). Why: Hybrid features such as seamless sign-in, password writeback, and Conditional Access require P1+. Verify: M365 admin center -> `Billing -> Licenses`; confirm Entra ID P1/P2 (often bundled in M365 E3/E5 or EMS). Implement: Assign M365 E3/E5, EMS E3/E5, or standalone Entra ID P1/P2.

G3. Administrative roles

Description: Hybrid Identity Administrator and Global Administrator for the operator. Why: Configuring directory synchronization and authentication methods needs these elevated roles. Verify: Entra -> `Roles and administrators`; confirm `Hybrid Identity Administrator` (and a Global Administrator for the one-time first-sync consent). Implement: Assign `Hybrid Identity Administrator`; reserve `Global Administrator` for tenant-wide and first-sync changes.

G4. Break-glass (emergency access) accounts

Description: At least two cloud-only emergency access accounts, excluded from Conditional Access and MFA-broad policies. Why: If sync, federation, or an auth-method change locks out admins, break-glass accounts get you back in. Verify: Entra -> `Users`; confirm two cloud-only accounts (`@onmicrosoft.com`) with strong stored credentials and documented use. Implement: Create two cloud-only Global Administrator accounts; exclude them from policies; store credentials securely and monitor sign-ins.

G5. Healthy on-premises Active Directory

Description: A functional AD forest/domain with healthy replication and DNS. Why: Synchronization reads from AD; replication or DNS faults corrupt or stall the sync. Verify: Run `dcdiag` and `repadmin /replsummary` on a domain controller with no errors; run `HYBID-Discovery.ps1`. Implement: Resolve AD replication, DNS, and time-sync issues before synchronizing.

G6. An Entra Connect host server

Description: A domain-joined Windows Server (2016+), patched, for Microsoft Entra Connect Sync (or a lightweight host with the Cloud Sync agent). Why: Connect Sync runs as a service on this host and must reach both AD and Entra ID. Verify: A supported, domain-joined server with TLS 1.2, .NET, and PowerShell available; run `HYBID-Discovery.ps1`. Implement: Provision a dedicated server (not a domain controller for production), patch it, and enable TLS 1.2.

G7. UPN / domain hygiene

Description: User principal names use a public, verified, routable domain (not `.local`). Why: A non-routable UPN suffix produces sign-in problems and `onmicrosoft.com` fallbacks in the cloud. Verify: Entra -> `Identity -> Settings -> Domain names` shows the public domain Verified; AD UPN suffixes match it; run `HYBID-Discovery.ps1`. Implement: Add and verify the public domain in Entra; add the matching UPN suffix in AD and update user UPNs.

G8. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS (443) from the Connect host to Entra ID, plus required Microsoft 365 service URLs. Why: Synchronization, authentication-agent traffic, and tokens depend on these endpoints.

Verify: From the Connect host, reach `https://login.microsoftonline.com` and the Microsoft 365 published URL/IP ranges. Implement: Allow the Microsoft 365 / Entra service URLs and IP ranges through the proxy/firewall.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Entra Verified ID — Verifiable Credentials

Follow these steps in order, in the **Microsoft Entra admin center** (`https://entra.microsoft.com`, under `Verified ID`) and the **Azure portal** (`https://portal.azure.com`) for Key Vault. Stand up the service, one credential type, one issuance flow, and one verification flow first; expand only after the Testing Guide passes.

> *Model: onboard the service and verify your domain, design one credential type, register the Request Service app, issue one credential to a wallet, verify it from a relying party, and revoke it — validated on a pilot before you expand. Verified ID feature/preview status is flagged *verify* — check current Microsoft documentation for your tenant before relying on any preview capability. Naming convention: every object uses* `<Purpose> – <Scope> – <Stage>`*. The pilot objects are:* `Verified ID Signing Key – Issuer – Pilot` *(Key Vault key, created during onboarding),* `Verified Employee – HR Attributes – Pilot` *(credential type),* `VC Request Service App – Issuer/Verifier – Pilot` *(app registration),* `Issuance Flow – Verified Employee – Pilot`*, and* `Verification Flow – Helpdesk Access – Pilot`*. Reuse these exact names in testing, troubleshooting, and the scripts. Replace* `contoso.com` *with your real domain.*

Step 1 — Assess prerequisites

1. Run `ENTVID-Discovery.ps1` to record whether Verified ID is available, whether an authority/issuer already exists, and the tenant/licensing context. Keep the output as your before-state.

2. Confirm the requirements: Azure Key Vault present, a host-controllable domain (`contoso.com`), and roles. Run `ENTVID-Validate.ps1` (some checks will legitimately fail until after onboarding — that is expected on the first pass).

Step 2 — Create/prepare the Azure Key Vault

1. Azure portal (`portal.azure.com`) -> create (or choose) a Key Vault in the same tenant.

2. Grant the operator and the Verified ID onboarding process the documented key permissions (get, sign, and key create) via access policies or Azure RBAC key roles. Follow current Verified ID onboarding guidance for the exact set. The signing key created here will be `Verified ID Signing Key – Issuer – Pilot`.

Step 3 — Onboard the Verified ID service and set up the trust system

1. Entra -> `Verified ID -> Setup` (or `Get started`).

2. Choose the trust system and complete onboarding; select the Key Vault from Step 2 when prompted. Onboarding provisions your organization's **DID** and signing keys.

3. Note the **issuer/authority** identifier shown after onboarding — the scripts and requests reference it. Run `ENTVID-Discovery.ps1` to confirm the authority now exists.

**Verify:* the trust system options and any preview features shown during onboarding — confirm against current Microsoft Verified ID documentation for your tenant before choosing.*

Step 4 — Verify your domain

1. In `Verified ID`, set/confirm the **linked domain** (`https://contoso.com`).

2. Download the generated `did-configuration.json` and publish it at `https://contoso.com/.well-known/did-configuration.json` on a web server you control.

3. Back in `Verified ID`, verify the domain; confirm it shows Verified. Run `ENTVID-Validate.ps1` to confirm the well-known file resolves and matches your DID.

Step 5 — Design the credential type (rules + display)

1. Entra -> `Verified ID -> Credentials -> Add a credential`.

2. Create `Verified Employee – HR Attributes – Pilot` (start from the Verified Employee template where suitable).

3. **Rules definition**: choose the attestation method — for a directory-backed credential, use the ID-token/Entra sign-in method so claims (name, job title, employment status) come from the signed-in user's directory record.

4. **Display definition**: set the card name, logo, colors, and claim labels shown in the wallet. Create/publish the credential type.

Step 6 — Register the Request Service API app

1. Entra -> `App registrations -> New registration`; name it `VC Request Service App – Issuer/Verifier – Pilot`.

2. `API permissions -> Add a permission` -> add the **Verified ID Request Service** application permission and **grant admin consent**. *Verify the current permission name/ID for your tenant.*

3. `Certificates & secrets` -> create a certificate (preferred) or client secret. Record the client ID, tenant ID, and store the secret/certificate securely (never in source control).

Step 7 — Issue a pilot credential (issuance flow)

1. Build the issuance request with `ENTVID-New-IssuanceRequest.ps1` in **build-only** mode (the default): it assembles and prints the issuance request payload for `Verified Employee – HR Attributes – Pilot` without submitting.

2. Review the payload; then re-run with `-Submit` to POST it to the Request Service API. The script returns a QR code URL / deep link.

3. On the pilot user's device, open Microsoft Authenticator, scan the QR code / follow the link, sign in and consent as the rules require; confirm the credential appears in the wallet. This is `Issuance Flow – Verified Employee – Pilot`.

Step 8 — Verify the pilot credential (verification flow)

1. Build the presentation (verification) request with `ENTVID-New-PresentationRequest.ps1` in **build-only** mode (default): it assembles the presentation request for `Verified Employee – HR Attributes – Pilot`, pointing at your callback endpoint, without submitting.

2. Review the payload; re-run with `-Submit` to POST it. The script returns a QR code / deep link.

3. As the pilot holder, scan/open it in Authenticator and consent to present the credential; confirm your relying-party callback receives the verified claims and cryptographic proof. This is `Verification Flow – Helpdesk Access – Pilot`.

Step 9 — Revoke and confirm governance

1. Entra -> `Verified ID -> Credentials`, open the issued credential, and **revoke** the pilot credential.

2. Re-run the verification flow (Step 8) and confirm a status-checking verifier now rejects the revoked credential. Run `ENTVID-Report.ps1` to see issued vs revoked state.

Step 10 — Validate on the pilot scope

1. Confirm end to end: domain verified, credential type published, issuance to wallet works, verification returns verified claims, revocation is honored. Run `ENTVID-Validate.ps1` and complete the Testing Guide.

Step 11 — Expand to production

1. Add more credential types (for example proof-of-training, contractor, or customer credentials) with their own rules/display, following the same pattern and naming.

2. Onboard more relying parties (verification flows), each with its own callback and requested claims.

3. Establish credential lifecycle policy (expiry, re-issuance cadence, revocation triggers) with your governance owners; keep the Request Service app's secret/certificate rotated.

4. Re-validate each change with `ENTVID-Validate.ps1` and `ENTVID-Report.ps1`.

Order of operations

```
Assess (Discovery/Validate) -> Key Vault -> Onboard service + DID -> Verify domain (well-known) -> Credential type (rules+display) -> Request Service app -> Issue to wallet (build then submit) -> Verify from relying party (build then submit) -> Revoke + confirm -> Validate -> Expand
```

5. Testing Guide — Entra Verified ID — Verifiable Credentials

Run after the pilot is configured (Implementation Step 10) and after each expansion. Do portal checks at `https://entra.microsoft.com` (Verified ID) and `https://portal.azure.com` (Key Vault); the discovery/validation scripts connect to Microsoft Graph, and the request-builder scripts call the Verified ID Request Service API only when you pass `-Submit`.

Test 0 — Service and prerequisites confirmed

Do: Run `ENTVID-Discovery.ps1` and `ENTVID-Validate.ps1`. Expected: Discovery shows Verified ID available and an issuer/authority onboarded; Validate confirms Key Vault access, the app registration with the Request Service permission, and the domain well-known file. A failure points to the exact prerequisite to fix.

Test 1 — Domain is verified (well-known resolves)

Do: Browse `https://contoso.com/.well-known/did-configuration.json`; in `Verified ID` confirm the linked domain shows Verified; or run `ENTVID-Validate.ps1`. Expected: The `did-configuration.json` resolves over HTTPS and links to your DID; the portal shows the domain Verified.

Test 2 — Credential type is published

Do: Entra -> `Verified ID -> Credentials`; confirm `Verified Employee – HR Attributes – Pilot` exists with rules and display; or run `ENTVID-Discovery.ps1`. Expected: The credential type is present and published, with the intended claims and display.

Test 3 — Issuance request builds safely (no submit)

Do: Run `ENTVID-New-IssuanceRequest.ps1` with no `-Submit`. Expected: The issuance request payload is assembled and printed; nothing is submitted to the Request Service API and no credential is issued (build-only is the default).

Test 4 — Credential issues to the wallet

Do: Re-run `ENTVID-New-IssuanceRequest.ps1 -Submit`; on the pilot device, scan the QR/deep link in Microsoft Authenticator and complete sign-in/consent. Expected: The credential `Verified Employee – HR Attributes – Pilot` appears in the wallet with the correct claims.

Test 5 — Verification returns verified claims

Do: Run `ENTVID-New-PresentationRequest.ps1 -Submit`; as the holder, present the credential in Authenticator; check the relying-party callback. Expected: The callback receives the verified claims and a valid cryptographic proof (issuer trusted, not tampered, not revoked). Consent was requested in the wallet and only the requested claims were shared.

Test 6 — Revocation is honored

Do: Revoke the pilot credential in `Verified ID -> Credentials`; re-run the verification flow (`ENTVID-New-PresentationRequest.ps1 -Submit`). Expected: A status-checking verifier rejects

the revoked credential; `ENTVID-Report.ps1` shows it as revoked.

Test 7 — Issued/revoked report

Do: Run `ENTVID-Report.ps1`. Expected: A clear picture of the credential types, issuance activity, and revoked credentials (subject to the reporting/logging available in your tenant), read-only.

Pass/fail gate

Gate	Requirement
Pilot pass	Test 0 confirms prerequisites; Tests 1-5 pass — domain verified, credential published, issuance builds safely then issues to a wallet, and verification returns verified claims with consent
Expand pass	Test 6 (revocation honored) and Test 7 (report) pass before adding more credential types and relying parties

6. Troubleshooting Guide — Entra Verified ID — Verifiable Credentials

Use `ENTVID-Discovery.ps1` / `ENTVID-Validate.ps1` for service and prerequisite state, `ENTVID-Report.ps1` for issued/revoked credentials, and the two request-builder scripts (`ENTVID-New-IssuanceRequest.ps1`, `ENTVID-New-PresentationRequest.ps1`, build-only unless `-Submit`) for the flows. Do portal work at `https://entra.microsoft.com` (Verified ID) and `https://portal.azure.com` (Key Vault).

A. Onboarding / service setup problems

Symptom	Cause	Remediation
Verified ID setup fails or Key Vault step errors	Operator or Verified ID service lacks Key Vault key permissions	Grant the documented Key Vault key permissions (get, sign, key create) via access policy or Azure RBAC key roles; retry onboarding; run `ENTVID-Validate.ps1`
No issuer/authority after onboarding	Onboarding not completed, or wrong tenant	Re-run `Verified ID -> Setup`; confirm you are in the correct tenant; run `ENTVID-Discovery.ps1`
`Verified ID` not visible in the admin center	Feature not available/licensed in the tenant	*Verify current Verified ID availability and licensing for your tenant*; confirm the required Entra ID licensing is assigned

B. Domain verification problems

Symptom	Cause	Remediation
Domain will not verify	`did-configuration.json` not published at `/.well-known/`, not HTTPS, or does not match the DID	Publish the generated file at `https://contoso.com/.well-known/did-configuration.json` over HTTPS; ensure it matches your DID; re-verify; run `ENTVID-Validate.ps1`
Well-known file returns 404	Wrong path or web server not serving `.well-known`	Confirm the exact `/.well-known/did-configuration.json` path and that the host serves dotfolder content
Domain shows Verified but wallets still warn	Cached/stale file or mismatched content	Re-download the current `did-configuration.json` from Verified ID and republish; clear caches

C. Request Service app / API problems

Symptom	Cause	Remediation
Request scripts get 401/403 from the Request Service API	Missing/unconsented API permission, wrong client credentials, or expired secret	Add and admin-consent the Verified ID Request Service permission; confirm client ID/tenant ID; rotate the secret/certificate. *Verify the current permission name/ID for your tenant*
Token acquisition fails	Wrong authority/scope in the client-credentials call	Confirm the token authority and the Request Service resource/scope used by the script; use the app's tenant ID
Secret found in logs/repo	Secret handled insecurely	Rotate the secret immediately; store it in a secrets store/env, never in source control or output

D. Issuance problems

Symptom	Cause	Remediation
QR / deep link never appears	Ran build-only (no `-Submit`)	This is expected in build-only mode; re-run `ENTVID-New-IssuanceRequest.ps1 -Submit` after reviewing the payload

Symptom	Cause	Remediation
Credential does not land in the wallet	Holder did not complete sign-in/consent, or attestation (rules) failed	Confirm the holder signs in as the rules require and consents; check the rules definition's attestation method; retry
Wrong/empty claims on the credential	Rules definition maps the wrong source claims	Fix the credential type's rules definition (claim mapping); re-issue
Wallet cannot scan / open link	No Authenticator or no connectivity	Install Microsoft Authenticator; ensure the device has connectivity; retry

E. Verification problems

Symptom	Cause	Remediation
Callback never receives a result	Callback endpoint not reachable by the Request Service, or wrong URL in the request	Use an HTTPS callback reachable from the internet; confirm the URL in `ENTVID-New-PresentationRequest.ps1`; re-submit
Verification rejects a valid credential	Credential expired or revoked, or issuer not trusted	Check issuance/expiry/revocation with `ENTVID-Report.ps1`; confirm the verifier trusts your issuer/DID and domain
Revoked credential still accepted	Verifier is not checking status	Ensure the verification request checks credential status/revocation; re-test after revoking (Step 9)

F. Governance / lifecycle

- Revocation: revoke in `Verified ID -> Credentials`; confirm a status-checking verifier rejects it. Run `ENTVID-Report.ps1` to see revoked state.
- Expiry/re-issuance: if credentials expire unexpectedly or never, review the credential type's lifecycle/expiry settings and your re-issuance cadence with your governance owners.

G. Diagnostics quick reference

```
# Read-only service/authority discovery via Microsoft Graph
Connect-MgGraph -Scopes 'VerifiableCredential.Authority.Read.All' -NoWelcome   # verify the exact scope fo
# Domain well-known check (no auth)
Invoke-WebRequest -UseBasicParsing "https://contoso.com/.well-known/did-configuration.json" | Select-Objec
# App registration + Request Service permission
Connect-MgGraph -Scopes 'Application.Read.All'
Invoke-MgGraphRequest -Method GET -Uri "/v1.0/applications?`$filter=displayName eq 'VC Request Service App
```

> *The Verified ID admin Graph scopes/endpoints and Request Service API details can differ by tenant and may be preview — *verify the current names/IDs for your tenant* before scripting against them.*

H. When to escalate

If Key Vault permissions are correct, the domain well-known file resolves and matches your DID, the Request Service app is consented with a valid credential, and issuance or verification still fails, capture `ENTVID-Validate.ps1` output and the Request Service API error/response and open a Microsoft support case for the Verified ID service, confirming whether any capability in play is preview for your tenant.

7. Best Practices — Entra Verified ID — Verifiable Credentials

• Protect the signing keys like the crown jewels. Your DID's signing keys in Azure Key Vault are the root of trust for every credential you issue. Restrict Key Vault access to the minimum, prefer Azure RBAC key roles, enable logging, and never export the keys.

• Verify your domain and keep the well-known file live. The `did-configuration.json` at `https://contoso.com/.well-known/` is how verifiers see a trustworthy, human-readable issuer. Monitor that it stays published and valid; a missing or stale file undermines trust in your credentials.

• Use a certificate, not a secret, for the Request Service app — and rotate it. The `VC Request Service App – Issuer/Verifier – Pilot` credential authorizes issuance and verification; prefer a certificate, store it securely (never in source control), and rotate on a schedule.

• Build requests before you submit them. The issuance and verification scripts default to build-only for a reason: review the assembled payload (claims requested, callback, PIN options) before you `-Submit`, so you never issue or request something unintended.

• Request the minimum claims. Privacy is a feature of Verified ID — design verification flows to request only the claims the decision needs (selective disclosure), and make consent explicit. Do not collect attributes you cannot justify.

• Design credential types deliberately. Give each credential a clear purpose, a correct attestation method (directory-backed vs self-attested vs presentation-backed), a sensible expiry, and clean display branding. Start from the Verified Employee pattern where it fits.

• Plan revocation and lifecycle up front. Decide what triggers revocation (offboarding, role change, compromise), set credential expiry, and define a re-issuance cadence — with your governance owners, not ad hoc.

• Keep the pilot to one credential type and one relying party. Prove issuance, verification, and revocation end to end on a small scope before expanding to more credentials and verifiers.

• Confirm the holder's wallet path. Ensure pilot holders have Microsoft Authenticator and can scan a QR / open a deep link before you run an issuance; most first-run failures are wallet/connectivity, not the service.

• Do not assume Verified ID is authoritative by itself. A credential proves what the issuer asserted; whether it is sufficient for a given access or onboarding decision is a governance decision you own.

• Verify feature and preview status for your tenant. Verified ID capabilities, premium options, and any preview features can change and vary by cloud — verify current availability and licensing against Microsoft documentation for your tenant before you depend on a capability.

• Name every object consistently. Use `<Purpose> – <Scope> – <Stage>` for keys, credential types, apps, and flows so audits and expansion are repeatable.

• Pilot before you expand, and keep a rollback path. Validate the full loop (issue -> verify -> revoke) before rollout; keep the ability to revoke and re-issue.

Where to go next (adjacent catalog areas)

• **Zero Trust Conditional Access (**`ENTZTCA`**)** — the Conditional Access baseline; explore using a Verified ID presentation as a signal/authentication context where supported (*verify current availability*).

• **Entra ID Governance (**`ENTIGA`**)** — entitlement management and access reviews that can consume a Verified ID-backed onboarding or access decision.

• **External Identities / B2B (**`ENTB2B`**)** — onboard and prove external users; Verified ID is a strong way to establish trust in a partner or contractor identity.

• **Passwordless / passkey (**`ENTPASSKEY`**)** and **Privileged Identity Management (**`ENTPIM`**)** — strengthen the authentication and privileged-access model around the identities you now verify.

• **Enterprise-app / service-principal governance** — govern the Request Service app and any relying-party apps you register.

8. Deliverables Summary — Entra Verified ID — Verifiable Credentials

Documentation

Document	Purpose
Executive Summary	Business case, outcome, scope boundary, and what is in the box
1. Solution Overview	What Verified ID is: trust system, DID, domain verification, credential types (rules + display), issuance and verification flows, the Request Service API, benefits, limitations, and how it plugs into `ENTZTCA` / `ENTIGA` / `ENTB2B` (gap-only)
2. Requirements Guide	Solution-specific prerequisites with Verify / Implement-if-missing, including Key Vault, domain, and app-registration items, with feature/preview/licensing flagged *verify*
Global Requirements (Entra ID Solutions)	Category-wide prerequisites (tenant, base licensing, admin roles, break-glass, network), published once
3. Implementation Guide	Exact, ordered steps — onboard the service and DID, verify the domain, design a credential type, register the Request Service app, issue to a wallet, verify from a relying party, and revoke — with every object named

Document	Purpose
5. Testing Guide	Validation procedures with pass/fail gates, including a build-only (no-submit) issuance test
6. Troubleshooting Guide	Symptom -> cause -> fix for the common failures
7. Best Practices	Field-proven recommendations and where to go next

PowerShell scripts (Microsoft Graph + Verified ID Request Service API)

Script	What it does
`ENTVID-Discovery.ps1`	Reports Verified ID availability, the onboarded issuer/authority, and published credential types (read-only)
`ENTVID-Validate.ps1`	Confirms prerequisites: Key Vault access, the Request Service app registration and permission, and the domain `did-configuration.json` well-known file (read-only)
`ENTVID-Report.ps1`	Issued/revoked credential and credential-type report for governance (read-only, subject to available tenant logging)
`ENTVID-New-IssuanceRequest.ps1`	Builds an issuance request payload and only POSTs it to the Request Service API when `-Submit` is passed (build-only by default)
`ENTVID-New-PresentationRequest.ps1`	Builds a presentation (verification) request payload and only POSTs it when `-Submit` is passed (build-only by default)

Discovery, Validate, and Report are read-only against Microsoft Graph / your domain. The two request-builder scripts call the Verified ID Request Service API **only when `-Submit` is supplied** — by default they assemble and print the payload so you can review it before anything is issued or requested.

What the buyer receives

Delivered as a **Complete Solution Package (PDF + Word + scripts)**:

- This implementation guide as a Word document and PDF (white-label).
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the standard model: Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

A working Verified ID capability — issue and verify tamper-evident digital credentials for onboarding, proof of employment, and access, privacy-respecting by design and revocable — proven on a pilot before you

scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

ENTVID-Discovery.ps1

Purpose: Reports the current Entra Verified ID posture: whether the Verifiable Credentials/Verified ID admin surface responds for the tenant, the onboarded issuer/authority (DID) if present, and the published credential types (contracts). Read-only — it changes nothing. Use it to capture the before-state and confirm the service is onboarded before designing credentials. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTVID-Discovery.ps1 Connects to Microsoft Graph. The Verified ID admin Graph scopes/endpoints can differ by tenant and may be preview — VERIFY the exact scope for your tenant. This script requests a read scope and degrades gracefully (prints guidance) if the admin surface is not available to it. EXPECTED RESULTS A summary of tenant context and, where the admin surface is reachable, the issuer authority and credential types. Pass = summary prints; a "not available" note means the admin API/scope must be verified for your tenant (use the Verified ID blade in the portal to confirm onboarding). .NOTES White-label. Read-only. Verified ID admin API/scopes are VERIFY-flagged (tenant/preview dependent). > [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\ENTVID-Discovery.ps1 Connects to Microsoft Graph. The Verified ID admin Graph scopes/endpoints can differ by tenant and may be preview — VERIFY the exact scope for your tenant. This script requests a read scope and degrades gracefully (prints guidance) if the admin surface is not available to it. EXPECTED RESULTS A summary of tenant context and, where the admin surface is reachable, the issuer authority and credential types. Pass = summary prints; a "not available" note means the admin API/scope must be verified for your tenant (use the Verified ID blade in the portal to confirm onboarding). .NOTES White-label. Read-only. Verified ID admin API/scopes are VERIFY-flagged (tenant/preview dependent). > [CmdletBinding()] param()

ENTVID-New-IssuanceRequest.ps1

Purpose: Assembles a Verified ID ISSUANCE request payload for a credential type and, ONLY when -Submit is passed, POSTs it to the Verified ID Request Service API to start an issuance (returns a QR/deep link for the holder's wallet). By DEFAULT it is BUILD-ONLY: it prints the payload and submits nothing, so a dry run never issues a credential. This is the safe preview gate for an operational action. Authenticates with the Request Service app's client credentials. HOW TO RUN Build-only (default): powershell -ExecutionPolicy Bypass -File .\ENTVID-New-IssuanceRequest.ps1 `-TenantId "<tenant-guid>" -ClientId "<app-guid>"` -Authority "<issuer-authority>" -CredentialType "VerifiedEmployee" `-Manifest "https://verifiedid.did.msidentity.com/v1.0/tenants/<t>/verifiableCredentials/contracts/<id>/manifest"` -CallbackUrl "https://your-app.contoso.com/api/issue/callback" Submit: add -Submit and provide the client secret via the VC_CLIENT_SECRET environment variable (never pass secrets on the command line / in source). VERIFY the Request Service API base URL, token scope, and payload schema for your tenant/cloud. EXPECTED RESULTS Build-only: the JSON issuance request is printed; nothing is sent. -Submit: the Request Service response (request id + QR/deep-link URL) is printed. Then complete issuance in Microsoft Authenticator. .NOTES White-label. Operational action GATED behind -Submit (build-only default). Secret read from env, never logged. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$TenantId, [Parameter(Mandatory=$true)][string]$ClientId, [Parameter(Mandatory=$true)][string]$Authority, [string]$CredentialType = 'VerifiedEmployee',

[Parameter(Mandatory=$true)][string]$Manifest, [Parameter(Mandatory=$true)][string]$CallbackUrl, [string]$CallbackState = ([guid]::NewGuid().ToString()), VERIFY these for your tenant/cloud before submitting: [string]$RequestServiceScope = '3db474b9-6a0c-4840-96ac-1fceb342124f/.default', [string]$RequestApiUrl = 'https://verifiedid.did.msidentity.com/v1.0/verifiableCredentials/createIssuanceRequest', [switch]$Submit)

How to run: Build-only (default): powershell -ExecutionPolicy Bypass -File .\ENTVID-New-IssuanceRequest.ps1 `-TenantId "<tenant-guid>" -ClientId "<app-guid>"` -Authority "<issuer-authority>" -CredentialType "VerifiedEmployee" `-Manifest "https://verifiedid.did.msidentity.com/v1.0/tenants/<t>/verifiableCredentials/contracts/<id>/manifest"` -CallbackUrl "https://your-app.contoso.com/api/issue/callback" Submit: add -Submit and provide the client secret via the VC_CLIENT_SECRET environment variable (never pass secrets on the command line / in source). VERIFY the Request Service API base URL, token scope, and payload schema for your tenant/cloud. EXPECTED RESULTS Build-only: the JSON issuance request is printed; nothing is sent. -Submit: the Request Service response (request id + QR/deep-link URL) is printed. Then complete issuance in Microsoft Authenticator. .NOTES White-label. Operational action GATED behind -Submit (build-only default). Secret read from env, never logged. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$TenantId, [Parameter(Mandatory=$true)][string]$ClientId, [Parameter(Mandatory=$true)][string]$Authority, [string]$CredentialType = 'VerifiedEmployee', [Parameter(Mandatory=$true)][string]$Manifest, [Parameter(Mandatory=$true)][string]$CallbackUrl, [string]$CallbackState = ([guid]::NewGuid().ToString()), VERIFY these for your tenant/cloud before submitting: [string]$RequestServiceScope = '3db474b9-6a0c-4840-96ac-1fceb342124f/.default', [string]$RequestApiUrl = 'https://verifiedid.did.msidentity.com/v1.0/verifiableCredentials/createIssuanceRequest', [switch]$Submit)

ENTVID-New-PresentationRequest.ps1

Purpose: Assembles a Verified ID PRESENTATION (verification) request payload for a relying party — asking a holder to present a credential of a given type from a trusted issuer — and, ONLY when -Submit is passed, POSTs it to the Verified ID Request Service API (returns a QR/deep link and, on presentation, a callback with the verified claims). By DEFAULT it is BUILD-ONLY: it prints the payload and submits nothing, so a dry run never sends a verification request. This is the safe preview gate. Authenticates with the Request Service app's client credentials. HOW TO RUN Build-only (default): powershell -ExecutionPolicy Bypass -File .\ENTVID-New-PresentationRequest.ps1 `-TenantId "<tenant-guid>" -ClientId "<app-guid>"` -CredentialType "VerifiedEmployee" `-AcceptedIssuer "<issuer-authority-or-DID>"` -CallbackUrl "https://your-app.contoso.com/api/verify/callback" Submit: add -Submit and provide the client secret via the VC_CLIENT_SECRET environment variable (never pass secrets on the command line / in source). VERIFY the Request Service API base URL, token scope, and payload schema for your tenant/cloud. EXPECTED RESULTS Build-only: the JSON presentation request is printed; nothing is sent. -Submit: the Request Service response (request id + QR/deep-link URL) is printed; when the holder presents, your callback receives the verified claims + proof. .NOTES White-label. Operational action GATED behind -Submit (build-only default). Secret read from env, never logged. Request only the claims the decision needs (selective disclosure / privacy by design). > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$TenantId, [Parameter(Mandatory=$true)][string]$ClientId, [string]$CredentialType = 'VerifiedEmployee', [Parameter(Mandatory=$true)][string]$AcceptedIssuer, [Parameter(Mandatory=$true)][string]$CallbackUrl, [string]$CallbackState = ([guid]::NewGuid().ToString()), [string]$Purpose = 'Verify employment for helpdesk identity proofing (Pilot)', VERIFY these for your tenant/cloud before submitting:

[string]$RequestServiceScope = '3db474b9-6a0c-4840-96ac-1fceb342124f/.default', [string]$RequestApiUrl = 'https://verifiedid.did.msidentity.com/v1.0/verifiableCredentials/createPresentationRequest', [switch]$Submit)

How to run: Build-only (default): powershell -ExecutionPolicy Bypass -File .\ENTVID-New-PresentationRequest.ps1 `-TenantId "<tenant-guid>" -ClientId "<app-guid>"` -CredentialType "VerifiedEmployee" `-AcceptedIssuer "<issuer-authority-or-DID>"` -CallbackUrl "https://your-app.contoso.com/api/verify/callback" Submit: add -Submit and provide the client secret via the VC_CLIENT_SECRET environment variable (never pass secrets on the command line / in source). VERIFY the Request Service API base URL, token scope, and payload schema for your tenant/cloud. EXPECTED RESULTS Build-only: the JSON presentation request is printed; nothing is sent. -Submit: the Request Service response (request id + QR/deep-link URL) is printed; when the holder presents, your callback receives the verified claims + proof. .NOTES White-label. Operational action GATED behind -Submit (build-only default). Secret read from env, never logged. Request only the claims the decision needs (selective disclosure / privacy by design). > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$TenantId, [Parameter(Mandatory=$true)][string]$ClientId, [string]$CredentialType = 'VerifiedEmployee', [Parameter(Mandatory=$true)][string]$AcceptedIssuer, [Parameter(Mandatory=$true)][string]$CallbackUrl, [string]$CallbackState = ([guid]::NewGuid().ToString()), [string]$Purpose = 'Verify employment for helpdesk identity proofing (Pilot)', VERIFY these for your tenant/cloud before submitting: [string]$RequestServiceScope = '3db474b9-6a0c-4840-96ac-1fceb342124f/.default', [string]$RequestApiUrl = 'https://verifiedid.did.msidentity.com/v1.0/verifiableCredentials/createPresentationRequest', [switch]$Submit)

ENTVID-Report.ps1

Purpose: Produces a governance report for Verified ID: the onboarded issuer authority, the published credential types (contracts), and — where the tenant exposes it — issuance/revocation signals from the Entra audit log (VerifiableCredential / Verified ID category). Read-only — it changes nothing. Optionally writes a CSV of the audit rows. Reporting depth depends on what your tenant logs and the admin API surface available, both of which are VERIFY-flagged. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTVID-Report.ps1 -Days 30 -CsvPath .\entvid-report.csv Connects to Microsoft Graph read scopes: AuditLog.Read.All and (best effort) VerifiableCredential.Authority.Read.All (VERIFY the current scope for your tenant). EXPECTED RESULTS The issuer authority + credential types, and a table of recent Verified ID audit events (issuance, revocation, config changes) over the last N days where available. Read-only. .NOTES White-label. Read-only. Verified ID admin API/scopes and audit categories are VERIFY-flagged. > [CmdletBinding()] param([int]$Days = 30, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\ENTVID-Report.ps1 -Days 30 -CsvPath .\entvid-report.csv Connects to Microsoft Graph read scopes: AuditLog.Read.All and (best effort) VerifiableCredential.Authority.Read.All (VERIFY the current scope for your tenant). EXPECTED RESULTS The issuer authority + credential types, and a table of recent Verified ID audit events (issuance, revocation, config changes) over the last N days where available. Read-only. .NOTES White-label. Read-only. Verified ID admin API/scopes and audit categories are VERIFY-flagged. > [CmdletBinding()] param([int]$Days = 30, [string]$CsvPath)

ENTVID-Validate.ps1

Purpose: Confirms the pilot prerequisites are in place and correct: the domain's /.well-known/did-configuration.json resolves over HTTPS (and, if an issuer DID is provided, that the file references it); the Request Service app registration exists with a credential and (best effort) the Verified ID Request Service API permission; and the named Key Vault is referenced. Read-only — it reports PASS/FAIL per check and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTVID-Validate.ps1 `-Domain "contoso.com"` -RequestServiceAppName "VC Request Service App - Issuer/Verifier - Pilot" ` -IssuerDid "did:web:contoso.com" # optional, to cross-check the well-known file Connects to Microsoft Graph read scope Application.Read.All for the app check; the domain check needs no auth. EXPECTED RESULTS PASS/FAIL lines for the well-known file, the app registration + credential, and (best effort) the API permission. Any FAIL points to the exact prerequisite to fix (see the Troubleshooting Guide). .NOTES White-label. Read-only. Request Service API permission ID is VERIFY-flagged (tenant/preview dependent). > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$Domain, [string]$RequestServiceAppName = 'VC Request Service App - Issuer/Verifier - Pilot', [string]$IssuerDid)

How to run: powershell -ExecutionPolicy Bypass -File .\ENTVID-Validate.ps1 `-Domain "contoso.com"` -RequestServiceAppName "VC Request Service App - Issuer/Verifier - Pilot" ` -IssuerDid "did:web:contoso.com" # optional, to cross-check the well-known file Connects to Microsoft Graph read scope Application.Read.All for the app check; the domain check needs no auth. EXPECTED RESULTS PASS/FAIL lines for the well-known file, the app registration + credential, and (best effort) the API permission. Any FAIL points to the exact prerequisite to fix (see the Troubleshooting Guide). .NOTES White-label. Read-only. Request Service API permission ID is VERIFY-flagged (tenant/preview dependent). > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$Domain, [string]$RequestServiceAppName = 'VC Request Service App - Issuer/Verifier - Pilot', [string]$IssuerDid)